Artificial Jewellery

E-Commerce Platform

Detailed Architecture & Design Document

Generated: December 25, 2025

Version 1.0

Table of Contents

1. Introduction 3

2. Tech Stack 4

3. High-Level Architecture 5

4. NestJS Backend Modules 6

5. Sanity CMS Content Schema 7

6. Database Schema (Prisma) 8

7. Product Flow & API Endpoints 9

8. Ordering Logic 10

9. Discount Logic 11

10. Payment Integration (Razorpay) 12

11. AI Recommendation Readiness 13

12. Future Extensions 14

13. Example Scenarios 15

14. Conclusion 16

1. Introduction

This document provides a comprehensive architecture and design specification for an Artificial Jewellery E-Commerce Platform. The platform is designed to be highly scalable, maintainable, and ready for future enhancements including AI-powered jewellery recommendations based on dress images. The architecture leverages modern technologies including NestJS for the backend, Next.js for the frontend, Sanity CMS for content management, PostgreSQL with Prisma ORM for data persistence, and Razorpay for payment processing. The system is designed with modularity and scalability as core principles, ensuring it can grow with business needs.

2. Tech Stack

Component	Technology	Purpose
Frontend Framework	Next.js 14+	Server-side rendering, React-based UI
Backend Framework	NestJS	Modular, scalable Node.js backend
Content Management	Sanity CMS	Headless CMS for product content
Database	PostgreSQL	Relational database for transactional data
ORM	Prisma	Type-safe database access
Payment Gateway	Razorpay	Payment processing
Future AI Module	Python/ML (TBD)	Image-based jewellery recommendations

3. High-Level Architecture

3.1 System Overview

The system follows a layered architecture pattern with clear separation of concerns: **Presentation Layer (Next.js Frontend):** Handles user interactions, displays product catalogs, manages shopping carts, and processes checkout flows. Implements server-side rendering for SEO and performance. **Application Layer (NestJS Backend):** Contains business logic, API endpoints, authentication, order processing, and integration with external services. Organized into feature modules for maintainability. **Data Layer:** Consists of PostgreSQL database (transactional data) and Sanity CMS (content data). Prisma ORM provides type-safe access to PostgreSQL. **External Services:** Razorpay for payments, Sanity CMS API for content, and future AI service for recommendations.

3.2 Architecture Diagram

The following diagram illustrates the high-level system architecture: [Frontend (Next.js)] ↓ HTTP/REST [Backend (NestJS)] ↓ ↓ ↓ [Sanity CMS] [PostgreSQL] [Razorpay] ↓ [AI Module (Future)] **Key Interactions:** • Frontend communicates with Backend via REST API • Backend fetches product content from Sanity CMS • Backend stores transactional data in PostgreSQL • Backend processes payments through Razorpay • Backend will integrate with AI module for recommendations (future)

4. NestJS Backend Modules

The NestJS backend is organized into feature modules following Domain-Driven Design principles. Each module is self-contained with its own controllers, services, and DTOs.

Module	Responsibility	Key Endpoints
Catalog	Product management, listings, search	GET /products, GET /products/:id, GET /categories
Cart	Shopping cart management	POST /cart, GET /cart, PUT /cart/:id, DELETE /cart/:id
Order	Order creation and management	POST /orders, GET /orders/:id, GET /orders
Payment	Payment processing via Razorpay	POST /payments/create, POST /payments/verify
User	User authentication and profiles	POST /auth/register, POST /auth/login, GET /user/profile
Admin	Administrative operations	POST /admin/products, PUT /admin/products/:id

4.1 Catalog Module Structure

Folder Structure:

```
src/catalog/
├── catalog.module.ts
├── catalog.controller.ts
├── catalog.service.ts
├── catalog.repository.ts
├── sanity-adapter.service.ts
├── dto/
│   ├── product-response.dto.ts
│   ├── variant-response.dto.ts
│   └── product-query.dto.ts
├── entities/
└── product.entity.ts
```

5. Sanity CMS Content Schema

Sanity CMS stores product content including descriptions, images, tags, and metadata. The backend merges this content with database records that contain business logic data (stock, prices, variants).

5.1 Product Schema

```
// schemas/product.js export default { name: 'product', title: 'Product', type: 'document', fields: [ { name: 'sanityId', title: 'Sanity ID', type: 'string', description: 'Unique identifier matching DB product.sanityId' }, { name: 'title', title: 'Product Title', type: 'string', required: true }, { name: 'description', title: 'Description', type: 'text', required: true }, { name: 'images', title: 'Product Images', type: 'array', of: [{ type: 'image' }] }, { name: 'category', title: 'Category', type: 'reference', to: [{ type: 'category' }] }, { name: 'tags', title: 'Tags', type: 'array', of: [{ type: 'string' }] }, { name: 'badge', title: 'Badge (e.g., "New", "Sale")', type: 'string' }, { name: 'displayRank', title: 'Display Rank (lower = higher priority)', type: 'number' } ] }
```

6. Database Schema (Prisma)

```
// prisma/schema.prisma model User { id String @id @default(cuid()) email String @unique password String name String? phone String? createdAt DateTime @default(now()) updatedAt DateTime @updatedAt orders Order[] cartItems CartItem[] } model Product { id String @id @default(cuid()) sanityId String @unique sku String @unique basePrice Decimal @db.Decimal(10, 2) stock Int @default(0) isActive Boolean @default(true) createdAt DateTime @default(now()) updatedAt DateTime @updatedAt variants Variant[] orderItems OrderItem[] cartItems CartItem[] } model Variant { id String @id @default(cuid()) productId String name String // e.g., "Gold", "Silver" price Decimal @db.Decimal(10, 2) // Overrides product.basePrice if set stock Int @default(0) sku String @unique discount Decimal? @db.Decimal(5, 2) // Percentage discount (variant-level) isActive Boolean @default(true) product Product @relation(fields: [productId], references: [id]) @@index([productId]) } model Category { id String @id @default(cuid()) name String @unique slug String @unique description String? parentId String? parent Category? @relation("CategoryHierarchy", fields: [parentId], references: [id]) children Category[] @relation("CategoryHierarchy") @@index([slug]) } model CartItem { id String @id @default(cuid()) userId String productId String variantId String? quantity Int @default(1) createdAt DateTime @default(now()) user User @relation(fields: [userId], references: [id]) product Product @relation(fields: [productId], references: [id]) @@unique([userId, productId, variantId]) } model Order { id String @id @default(cuid()) userId String status OrderStatus @default(PENDING) totalAmount Decimal @db.Decimal(10, 2) subtotal Decimal @db.Decimal(10, 2) discount Decimal @db.Decimal(10, 2) @default(0) shippingCost Decimal @db.Decimal(10, 2) @default(0) createdAt DateTime @default(now()) updatedAt DateTime @updatedAt user User @relation(fields: [userId], references: [id]) orderItems OrderItem[] payment Payment? shippingInfo ShippingInfo? } model OrderItem { id String @id @default(cuid()) orderId String productId String variantId String? quantity Int price Decimal @db.Decimal(10, 2) // Price at time of order discount Decimal @db.Decimal(5, 2) // Applied discount percentage order Order @relation(fields: [orderId], references: [id]) product Product @relation(fields: [productId], references: [id]) } model Payment { id String @id @default(cuid()) orderId String @unique razorpayId String? @unique amount Decimal @db.Decimal(10, 2) status PaymentStatus @default(PENDING) method String? createdAt DateTime @default(now()) updatedAt DateTime @updatedAt order Order @relation(fields: [orderId], references: [id]) } model ShippingInfo { id String @id @default(cuid()) orderId String @unique name String address String city String state String pincode String phone String order Order @relation(fields: [orderId], references: [id]) } enum OrderStatus { PENDING CONFIRMED PROCESSING SHIPPED DELIVERED CANCELLED } enum PaymentStatus { PENDING SUCCESS FAILED REFUNDED }
```

7. Product Flow & API Endpoints

Product Listing Flow:
1. Frontend requests product list: GET /api/catalog/products
2. Backend queries PostgreSQL for active products with stock > 0
3. For each product, fetch content from Sanity CMS using sanityId
4. Merge database data (price, stock) with Sanity content (images, description)
5. Apply ordering logic (in-stock first, then displayRank, then createdAt)
6. Return paginated results

7.1 API Response Example

```
// GET /api/catalog/products?page=1&limit;=20&category;=necklaces { "data": [
{ "id": "prod_123", "sanityId": "sanity_prod_123", "title": "Elegant Gold
Necklace", "description": "Beautiful artificial gold necklace...", "images":
[ "https://cdn.sanity.io/images/...", "https://cdn.sanity.io/images/..." ],
"basePrice": 1299.00, "stock": 45, "category": { "id": "cat_1", "name":
"Necklaces", "slug": "necklaces" }, "variants": [ { "id": "var_1", "name":
"Gold", "price": 1299.00, "stock": 30, "discount": null }, { "id": "var_2",
"name": "Rose Gold", "price": 1399.00, "stock": 15, "discount": 10.0 } ],
"badge": "New", "displayRank": 1, "tags": ["trending", "elegant"] } ],
"pagination": { "page": 1, "limit": 20, "total": 150, "totalPages": 8 } }
```

8. Ordering Logic

Products are ordered using a multi-level sorting strategy to ensure the best user experience:

```
Sorting Priority (in order):
1. Stock Availability: Products with stock > 0 appear first
2. Display Rank: Lower displayRank values appear first (1, 2, 3...)
3. Newness: Most recently created products appear first Pseudocode:
products.sort((a, b) => {
// Priority 1: Stock availability
if (a.stock > 0 && b.stock === 0) return -1;
if (a.stock === 0 && b.stock > 0) return 1;

// Priority 2: Display rank (if set)
if (a.displayRank !== null && b.displayRank !== null) {
return a.displayRank - b.displayRank;
}
if (a.displayRank !== null) return -1;
if (b.displayRank !== null) return 1;

// Priority 3: Newness
return b.createdAt - a.createdAt;
}); Database Query (Prisma):
const products = await prisma.product.findMany({
where: { isActive: true },
orderBy: [
{ stock: 'desc' }, // In-stock first
{ displayRank: 'asc' }, // Lower rank first
{ createdAt: 'desc' } // Newest first
],
take: limit,
skip: (page - 1) * limit
});
```

9. Discount Logic

Discounts are applied hierarchically with clear precedence rules:

```
Discount Hierarchy:
1. Variant-level discount (highest priority) - Overrides product-level
2. Product-level discount - Applied if no variant discount exists
3. No discount - Full price if no discounts set Rules:
• Only one discount can be active at a time
• Discounts are stored as percentages (e.g., 10.0 = 10%)
• Final price calculation: price * (1 - discount/100)
• Discounts are applied at cart calculation time Price Calculation
Pseudocode:
function calculatePrice(product, variant = null) {
let basePrice = product.basePrice;
let discount = null;

if (variant) {
basePrice = variant.price || product.basePrice;
discount = variant.discount || product.discount || null;
} else {
discount = product.discount || null;
}

if (discount) {
return basePrice * (1 - discount / 100);
}
return basePrice;
} Example:
Product base price: ■1,299
Variant "Rose Gold" price: ■1,399 (overrides base)
Variant discount: 10%
Final price: ■1,399 * 0.9 = ■1,259.10
```

10. Payment Integration (Razorpay)

Razorpay integration handles payment processing with webhook support for payment status updates.

```
Payment Flow:
1. User initiates checkout: POST /api/orders
2. Backend creates order with status PENDING
3. Backend creates Razorpay order: POST /api/payments/create
4. Razorpay returns order_id and key
5. Frontend redirects to Razorpay payment page
6. User completes payment on Razorpay
7. Razorpay webhook: POST /api/payments/webhook
8. Backend verifies payment signature
9. Backend updates order status to CONFIRMED
10. Backend updates payment record with razorpayId and status Webhook
Security:
• Verify Razorpay signature using secret key
• Validate order amount matches
• Idempotent processing (check if already processed)
```

11. AI Recommendation Readiness

The architecture is designed to accommodate future AI integration for image-based jewellery recommendations. The system will accept dress images and suggest matching jewellery pieces.

Planned AI Module Architecture:
1. **Image Upload Endpoint:** POST /api/ai/recommend
• Accepts image file (dress photo)
• Validates image format and size
• Stores temporarily or sends to AI service **AI Service Integration:**
• Separate microservice (Python/ML framework)
• Receives image, analyzes color, style, occasion
• Returns recommended product IDs with confidence scores
• Backend queries catalog for recommended products
• Returns enriched product data to frontend **Database Considerations:**
• Add aiRecommendations table to track recommendations
• Store image metadata, user preferences
• Track recommendation performance for ML improvement **Future Schema Addition:**

```
model AIRecommendation {
id String @id @default(cuid())
userId String?
imageUrl String
productIds String[] // Array of recommended product IDs
scores Float[] // Confidence scores
createdAt DateTime @default(now())
}
```

12. Future Extensions

Scalability Enhancements:
• Implement Redis caching for product listings
• Add Elasticsearch for advanced search capabilities
• Microservices architecture for independent scaling
• CDN integration for image delivery
• Queue system (Bull/Redis) for async order processing **Feature Extensions:**
• User reviews and ratings
• Wishlist functionality
• Order tracking with shipment updates
• Email/SMS notifications
• Admin dashboard for inventory management
• Analytics and reporting
• Multi-language support
• Multi-currency support **AI/ML Enhancements:**
• Personalized product recommendations
• Image-based search (find similar products)
• Chatbot for customer support
• Price optimization using ML
• Fraud detection

13. Example Scenarios

Scenario 1: User Browsing Products
1. User visits homepage: GET /api/catalog/products?page=1
2. Backend queries: SELECT * FROM products WHERE isActive = true AND stock > 0
3. For each product, fetch from Sanity: GET /v1/data/query?query=*[_id=="sanity_prod_123"]
4. Merge data and sort: in-stock → displayRank → createdAt
5. Return 20 products per page
6. Frontend displays product cards with images, prices, badges

Scenario 2: Adding to Cart with Discount
1. User selects product "Gold Necklace" with variant "Rose Gold"
2. Backend calculates price:
• Base price: ■1,399 (variant override)
• Discount: 10% (variant-level)
• Final price: ■1,259.10
3. User adds to cart: POST /api/cart { productId, variantId, quantity: 2 }
4. Cart total: ■2,518.20
5. Cart displays original price (■2,798), discount (10%), final price (■2,518.20)

Scenario 3: Order Placement & Payment
1. User proceeds to checkout: POST /api/orders { cartItems, shippingInfo }
2. Backend creates order: status = PENDING, totalAmount = ■2,518.20
3. Backend creates Razorpay order: amount = ■2,518.20
4. Razorpay returns: { order_id: "order_xyz", key: "rzp_test_..." }
5. Frontend redirects to Razorpay payment page
6. User pays ■2,518.20
7. Razorpay webhook: { razorpay_order_id, razorpay_payment_id, signature }
8. Backend verifies signature, updates order: status = CONFIRMED
9. Backend updates payment: status = SUCCESS, razorpayId = "pay_xyz"
10. User receives order confirmation

14. Conclusion

This architecture provides a solid foundation for an artificial jewellery e-commerce platform with the following key strengths: **Scalability:** Modular NestJS backend allows independent scaling of components. Database indexing and caching strategies support high traffic. **Maintainability:** Clear separation of concerns, type-safe Prisma ORM, and well-organized modules make the codebase maintainable. **Flexibility:** Sanity CMS enables content updates without code deployment. Discount and ordering logic can be easily modified. **Future-Ready:** Architecture accommodates AI integration, additional payment gateways, and feature extensions without major refactoring. **Next Steps:**
1. Set up development environment (NestJS, Next.js, PostgreSQL, Sanity)
2. Implement core modules (Catalog, Cart, Order, Payment)
3. Create Sanity CMS schemas and content
4. Set up Razorpay integration
5. Build frontend components
6. Testing and deployment
7. Future: Integrate AI recommendation module